PEMROGRAMAN BERBASIS FRAMEWORK
JOBSHEET 16

NAMA : Muhammad Fahreza Rohmansyah
KELAS : 3E
NO. ABSEN : 15
NIM : 2341720151

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026

Link Github:

https://github.com/rezfahreza/PBF_2026.git

Praktikum

1. Custom Login Page

- Modifikasi [...nextauth].ts

```
JS_16 > my-app > src > pages > api > auth > TS [...nextauth].ts > default
4   export const authOptions: NextAuthOptions = {
33    callbacks: {
41      async session({ session, token }:any) {
48        return session
49      },
50    },
51
52    pages: {
53      signIn: "/auth/login",
54    }
55  };
56
57  export default NextAuth(authOptions)
```

2. Handle Login di Frontend

- Modifikasi view/auth/login/index.tsx

```
import React, { useState } from "react";
import Link from "next/link";
import { useRouter } from "next/router";
import style from "./login.module.scss";
import { signIn } from "next-auth/react";

const TampilanLogin = () => {
    const [isLoading, setIsLoading] = useState(false);
    const [error, setError] = useState("");
    const { push, query } = useRouter();
    const callbackUrl: any = query.callbackUrl || "/";

    const handleSubmit = async (event: any,) => {
        event.preventDefault();
        setError("");
        setIsLoading(true);

        try {
            const res = await signIn("credentials", {
                redirect: false,
                email: event.target.email.value,
                password: event.target.password.value,
                callbackUrl,
            });

            if (res?.error) {
                setIsLoading(false);
                push(callbackUrl);
            } else {
                setIsLoading(false);
                setError(res?.error || "Login failed");
            }
        } catch (error) {
            setIsLoading(false);
            setError("wrong email or password");
        }

    };
```

```
    return (
        <div className={style.login}>
            <h1 className={style.login__title}>Halaman Login</h1>
            <div className={style.login__form}>
                <form onSubmit={handleSubmit}>
                    <div className={style.login__form__item}>
                        <label
                            htmlFor="email"
                            className={style.login__form__item__label}
                        >
                            Email
                        </label>
                        <input
                            type="email"
                            id="email"
                            name="email"
                            placeholder="Email"
                            className={style.login__form__item__input}
                        />
                    </div>
                    <div className={style.login__form__item}>
                        <label
                            htmlFor="password"
                            className={style.login__form__item__label}
                        >
                            Password
                        </label>
                        <input
                            type="password"
                            id="password"
                            name="password"
                            placeholder="Password"
                            className={style.login__form__item__input}
                        />
                    </div>
                    {error && <p className={style.login__error}>{error}</p>}
                    <button
                        type="submit"
                        className={style.login__form__item__button}
                        disabled={isLoading}
                    >
                        {isLoading ? "Loading..." : "Login"}
                    </button>
                </form>
                <br />
                <p className={style.login__form__item__text}>
                    Sudah punya akun? <Link href="/auth/login">Ke Halaman Login</Link>
                </p>
            </div>
        </div>
    );
};

export default TampilanLogin;
```

- Modifikasi login.module.scss

```
my-app > src > view > auth > login > login.module.scss > {} @media (max-width: 768px) >
.login {
    min-height: 100vh;
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    background: linear-gradient(135deg, #1e3c72, #2a5298);
    padding: 20px;

    &__title {
        width: 100%;
        text-align: center;
        font-size: 36px;
        font-weight: 700;
        color: #ffffff;
        margin-bottom: 40px;
    }

    &__form {
        background: #ffffff;
        width: 100%;
        max-width: 420px;
        padding: 40px;
        border-radius: 16px;
        box-shadow: 0 20px 40px rgba(0, 0, 0, 0.15);

        &__item {
            display: flex;
            flex-direction: column;
            margin-bottom: 20px;

            &__label {
                font-size: 14px;
                font-weight: 600;
                margin-bottom: 8px;
                color: #333;
            }

            &__input {
                padding: 12px 14px;
                border-radius: 10px;
                border: 1px solid #ddd;
                font-size: 14px;
                transition: all 0.3s ease;

                &:focus {
                    outline: none;
                    border-color: #2a5298;
                    box-shadow: 0 0 3px rgba(42, 82, 152, 0.2);
                }
            }
        }
```

```
        button {
            width: 100%;
            padding: 12px;
            border: none;
            border-radius: 10px;
            background: #2a5298;
            color: #ffffff;
            font-weight: 600;
            cursor: pointer;
            transition: 0.3s ease;

            &:hover {
                background: #1e3c72;
                transform: translateY(-2px);
            }
        }
    }
    &__error {
        color: #e74c3c;
        font-size: 14px;
        margin-top: 8px;
    }
}

/* Animation */
@keyframes fadeIn {
    from {
        opacity: 0;
        transform: translateY(15px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
}

/* Responsive */
@media (max-width: 768px) {
    .login {
        &__form {
            padding: 30px 20px;
        }

        &__title {
            font-size: 26px;
        }
    }
}
```

- Modifikasi servicefirebase.ts

```
export async function signIn( email: string ) {
    const q = query(collection(db, "users"), where("email", "==", email));
    const querySnapshot = await getDocs(q);
    const data = querySnapshot.docs.map((doc) => ({ id: doc.id, ...doc.data() }));
    if (data) {
        return data[0];
    } else {
        return null;
    }
}

export async function signUp(...
}
```

3. Authorize di NextAuth (Database Login)

- Modifikasi [...nextauth].ts

```
providers: [
  CredentialsProvider({
    name: "credentials",
    credentials: {
      //fullname: { label: "Full Name", type: "text" },
      email: { label: "Email", type: "email" },
      password: { label: "Password", type: "password" }
    },
    async authorize(credentials) {
      if (!credentials?.email || !credentials?.password) {
        return null
      }

      const user : any = await signIn(credentials.email);

      if (user) {
        const isValidPassword = await bcrypt.compare(
          credentials.password,
          user.password
        );

        if (isValidPassword) {
          return {
            id: user.id,
            email: user.email,
            fullname: user.fullname,
            role: user.role || "user"
          }
        }
      }
      return null
    }
  })
],
```

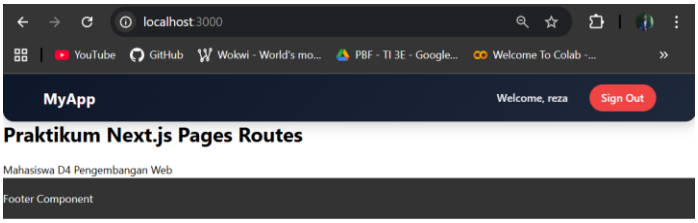
4. Tambahkan Role ke Token

- Modifikasi [...nextauth].ts

```
callbacks: {
  async jwt({ token, account, profile, user }:any) {
    if (account?.provider === "credentials" && user)
      token.email = user.email;
      token.fullname = user.fullname;
      token.role = user.role;
    }
    return token
  },
  async session({ session, token }:any) {
    if (token.email) {
      session.user.email = token.email
    }
    if (token.fullname) {
      session.user.name = token.fullname
    }
    if (token.role) {
      session.user.role = token.role
    }

    return session
  },
},
```

- Jalankan browser



5. Callback URL Logic

- Modifikasi withAuth.ts

```
my-app > src > middleware > TS withAuth.ts > withAuth > <function>
import { getToken } from "next-auth/jwt";
import { NextFetchEvent, NextMiddleware, NextRequest, NextResponse } from "next/server";

export default function withAuth(
  middleware: NextMiddleware,
  requireAuth: string[] = [],
) {
  return async (req: NextRequest, next: NextFetchEvent) => {
    const pathname = req.nextUrl.pathname;

    if (requireAuth.includes(pathname)) {
      const token = await getToken({
        req,
        secret: process.env.NEXTAUTH_SECRET,
      });

      if (!token) {
        const Url = new URL("/", req.url);
        Url.searchParams.set("callbackUrl", req.url);
        return NextResponse.redirect(Url);
      }
    }
    return middleware(req, next);
  };
}
```

6. Membuat halaman Admin dan authoriz

- Buat file pages/admin/index.tsx dan modifikasi

```
my-app > src > pages > admin > index.tsx > default
const HalamanAdmin = () => {
    return (
        <div>
            <div classNAme="admin">
                <h1>Halaman Admin</h1>
                <p>Selamat datang di halaman admin!,
                    anda adalah admin memiliki akses penuh ke semua fitur sistem.
                </p>
            </div>
        </div>
    );
}

export default HalamanAdmin;
```

- Modifikasi withAuth.ts

```
import { getToken } from "next-auth/jwt";
import { NextFetchEvent, NextMiddleware, NextRequest, NextResponse } from "next/server";

const hanyaAdmin = ["/admin"];

export default function withAuth(
  middleware: NextMiddleware,
  requireAuth: string[] = [],
) {
  return async (req: NextRequest, next: NextFetchEvent) => {
    const pathname = req.nextUrl.pathname;

    if (requireAuth.includes(pathname)) {
      const token = await getToken({
        req,
        secret: process.env.NEXTAUTH_SECRET,
      });

      if (!token) {
        const Url = new URL("/", req.url);
        Url.searchParams.set("callbackUrl", req.url);
        return NextResponse.redirect(Url);
      }
      if (token.role !== "admin" && hanyaAdmin.includes(pathname)) {
        return NextResponse.redirect(new URL("/", req.url));
      }
    }
    return middleware(req, next);
  }
}
```

Tugas Praktikum

1. Implementasikan login database.

2. Tambahkan role pada user.

3. Buat halaman:

- /profile

```
16 > my-app > src > pages > profile > index.tsx > ...
import { useSession } from "next-auth/react";

const HalamanProfile = () => {
    const { data }: { data: any } = useSession();

    return (
        <div>
            <h1>Halaman Profile</h1>
            <h1>Selamat Datang, {data?.user?.fullname}</h1>
        </div>
    );
};

export default HalamanProfile;
```

- /admin

```
5 > my-app > src > pages > admin > index.tsx > default
const HalamanAdmin = () => {
    return (
        <div>
            <div className="admin">
                <h1>Halaman Admin</h1>
                <p>Selamat datang di halaman admin!,
                    anda adalah admin memiliki akses penuh ke semua fitur sistem.
                </p>
            </div>
        </div>
    );
}

export default HalamanAdmin;
```

4. Proteksi /admin hanya untuk admin.

```
}
if (token.role !== "admin" && hanyaAdmin.includes(pathname)) {
  return NextResponse.redirect(new URL("/", req.url));
}
```

5. Implementasikan callback URL.

Pertanyaan Refleksi

1. Mengapa password harus diverifikasi dengan bcrypt.compare?

Karena password dalam database disimpan dalam bentuk hash yang tidak bisa dibaca langsung (dekripsi). bcrypt.compare digunakan untuk mencocokkan password yang diinput user dengan hash yang ada di database secara aman

2. Mengapa role disimpan di token?

Agar informasi peran (role) user dapat diakses dengan cepat oleh sistem (seperti middleware atau frontend) tanpa harus terus-menerus melakukan query ke database setiap kali user berpindah halaman. Token ini kemudian digunakan untuk menentukan hak akses user dalam satu sesi

3. Apa fungsi callbackUrl?

Berfungsi untuk menyimpan alamat URL halaman terakhir yang ingin diakses user sebelum dipaksa login. Setelah login berhasil, sistem akan menggunakan callbackUrl tersebut untuk mengarahkan user kembali ke halaman yang dituju sebelumnya

4. **Mengapa middleware penting untuk security?**

Middleware bertindak sebagai penjaga gerbang otomatis yang memeriksa status autentikasi user sebelum permintaan mencapai halaman atau data sensitif. Hal ini memastikan bahwa hanya user yang sah yang dapat melanjutkan proses ke sistem internal

5. **Apa risiko jika role tidak dicek di middleware?**

Tanpa pengecekan di middleware, user dengan level rendah (biasa) dapat mengakses halaman atau fitur administratif hanya dengan mengetikkan alamat URL-nya secara langsung. Hal ini membahayakan integritas data karena pihak yang tidak berwenang bisa mengelola atau menghapus data sensitif